

AI-Assisted Software Development

aider Cheat Sheet

Mike Burton 4-6 November 2025

Aider Commands - Grouped by Function:

- Context Management - Files
- Code Interaction Commands
- Version Control Commands
- Chat and Context Management
- Session Management Commands
- Model and Configuration
- Utility Commands
- Command Usage Patterns
- Command Relationships

Context Management - Files

These commands control which files are included in the LLM's context:

Command	Description
<code>/add</code>	Add files to the chat so aider can edit them or review them in detail
<code>/drop</code>	Remove files from the chat session to free up context space
<code>/ls</code>	List all known files and indicate which are included in the chat session
<code>/read-only</code>	Add files to the chat that are for reference, not to be edited
<code>/reset</code>	Drop all files and clear the chat history
<code>/web</code>	Scrape a webpage, convert to markdown and send in a message

Code Interaction Commands

These commands are for working with and analysing code:

Command	Description
<code>/architect</code>	Enter architect mode to discuss high-level design and architecture
<code>/ask</code>	Ask questions about the code base without editing any files
<code>/code</code>	Ask for changes to your code
<code>/lint</code>	Lint and fix in-chat files or all dirty files if none in chat
<code>/map</code>	Print out the current repository map
<code>/map-refresh</code>	Force a refresh of the repository map

Version Control Commands

These commands interact with Git:

Command	Description
<code>/commit</code>	Commit edits to the repo made outside the chat (commit message optional)
<code>/diff</code>	Display the diff of changes since the last message
<code>/git</code>	Run a git command (output excluded from chat)
<code>/undo</code>	Undo the last git commit if it was done by aider

Session Management Commands

These commands control the aider session:

Command	Description
<code>/exit</code>	Exit the application
<code>/quit</code>	Exit the application
<code>/load</code>	Load and execute commands from a file
<code>/save</code>	Save commands to a file that can reconstruct the current chat session's files
<code>/settings</code>	Print out the current settings

Chat and Context Management

These commands manage the conversation and its history:

Command	Description
/chat-mode	Switch to a new chat mode
/clear	Clear the chat history
/copy	Copy the last assistant message to the clipboard
/paste	Paste image/text from the clipboard into the chat. Optionally provide a name for the image.
/voice	Record and transcribe voice input
/tokens	Report on the number of tokens used by the current chat context

Model and Configuration

These commands manage LLM settings:

Command	Description
<code>/model</code>	Switch to a new LLM
<code>/models</code>	Search the list of available models

Utility Commands

These commands provide additional functionality:

Command	Description
<code>/help</code>	Ask questions about aider
<code>/report</code>	Report a problem by opening a GitHub Issue
<code>/run</code>	Run a shell command and optionally add the output to the chat (alias: !)
<code>/test</code>	Run a shell command and add the output to the chat on non-zero exit code

Command Usage Patterns

Common workflows often combine commands in specific ways:

1. File Management Workflow:

```
/ls           # Check current files  
/add src/*.py # Add source files  
/read-only docs/* # Add reference docs  
/tokens       # Check context size  
/drop unused.py # Free up space if needed
```

2. Code Review Workflow:

```
/add file.py   # Add file to review  
/ask           # Ask about implementation  
/diff          # Check specific changes  
/commit        # Commit any fixes
```

Command Usage Patterns (continued)

3. Architecture Discussion:

```
/architect      # Enter architecture mode  
/map            # Review project structure  
/map-refresh    # Update if needed
```

4. Model Optimization:

```
/models         # Check available models  
/model gpt-4    # Switch to specific model  
/tokens         # Monitor context usage
```

Command Relationships

